# Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

| Role | Name | Email |
|---|---|---|
| **Developer (You)** | Elinor Handel | ehandel@udel.edu |
| **Friend (optional)** | Alyssa Ebner | akebner@udel.edu |

## Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - Maybe I could have an LLM help me make the image through making a scatter plot image because it could probably map it out better than me

*I think it could be cool to help people realize if they are color blind by making a color blind test.*

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

| |
|---|
| *There are many websites that address red/green color blindness.* |

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

| |
|---|
| *I could make a website that is about other types of color blindness that is not addressed as much like blue yellow and total colorblindness.* |

# Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

| |
|---|
| The code will ask for an input of what color blindness they want to test and then provide the subsequent tests. It will output based on the score if the person should see an eye doctor as they might be color blind. |

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

| |
|---|
| They will have to input which type of colorblindness they think they could have. |

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

| |
|---|
| It will calculate the score of the test. |

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

Based on the user's score they will be told to visit an eye doctor or not.

# Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

*It needs to track the pictures and what numbers/symbols/letters are on them, it will also have to keep the amount of correct responses.*

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

*I will need the score, the color blind test colors, the symbols used, the amount of correct responses.*
*I will ignore invalid inputs, properties that only pertain to the other tests not being used in the current test.*

# Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

*It will be difficult to create the dotted image of the symbols with a scatter plot.*

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

*I can have an llm help me map out how to create the images to show for the test.*

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

*I want to help people who might have colorblindness realize if they do, it can be an issue that they struggle with that people who can see color don't think about.*

# Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

*I want to build a simple web tool that helps people realize if they have certain types of color blindness. What are some basic features this design should have?*

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

*Chat said that the website needs to be composed of the test with multiple types of the test, there should be an answer selection or type box, instant feedback, a results page, and a retake option.*

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

*It wanted to include a progress bar. I have no ideas on how to make a progress bar and think that it could be too difficult and time consuming to code. I could make a #/number of ?s just appear at the top instead and update the number with a count function.*

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

*This has allowed me to become more focused on the project and figure out more of the minute details.*

# Part 6: Resource Planning

1. What resources will you use if you get stuck?

*If I get stuck I can ask a TA, go to the Discord, attend office hours, look at the blockPy book, ask a friend, or ask an LLM on their suggestions without giving me a coded answer.*

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

*I will firstly look to see if I can recognize the problem and if not I will paste the error code into the LLM's chat box and see if it provides a fixed code that works, if that does not work then I will ask a friend, a TA, the discord, or go to office hours for help.*

# Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

*I want to build a simple web tool that helps people realize if they have certain types of color blindness. What are some basic features this design should have?*

A simple color blindness testing web tool should be easy to navigate, visually clear, and educational. The goal is to help users recognize whether they might have difficulty

distinguishing certain colors.

Here are some basic features I would recommend:

## 1. Main features

| Feature | Description |
| --- | --- |
| Color vision test | Show images with numbers or shapes hidden inside colored dots, similar to an Ishihara test. |
| Multiple test types | Include tests for red-green, blue-yellow, and other color vision deficiencies. |
| Answer selection | Allow users to type the number they see or select an answer from multiple choices. |
| Instant feedback | Tell users whether their answer was correct after each question. |
| Results page | Summarize their answers and indicate possible difficulty distinguishing certain colors. |
| Retake option | Allow users to restart the test and try again. |

## 2. Design and user experience

Color Vision Screening (Ishihara Test)

Color test images

Use colored dot patterns containing hidden numbers or shapes that users must identify.

Responsive frontend quiz app coding challenge solution

Simple interface

Use large buttons, readable text, and clear instructions so users can easily complete the test.

Minimal Onboarding / Questionnaire by Stian on Dribbble

Progress indicator

Show users how many questions they have completed and how many remain.

StudyLift — AI-powered learning for students

Results summary

Display their score and explain what their answers might indicate.

## 3. Educational features

You could also include:

- Types of color blindness: Explain protanopia, deuteranopia, tritanopia, and other color vision deficiencies.
- Examples: Show how everyday objects might appear differently to someone with color vision deficiency.
- Helpful resources: Provide links to professional eye-care resources.
- Testing disclaimer: Explain that the website is for educational screening purposes and is not a medical diagnosis.

## 4. Suggested website layout

I would organize the website into four simple sections:

# Color Vision Checker

Discover how you see color.

Start Color Test

1. Home page

Introduction and button to start the test.

2. Testing page

Display color plates, answer choices, and a progress bar.

3. Results page

Show the score and possible color vision difficulties.

4. Learn more page

Explain different types of color blindness and provide resources.

## 5. Important things to consider

- Make sure the test images are accurate and properly licensed.
- Include instructions to adjust screen brightness and avoid color filters or night mode.
- Make the website accessible to people who already have color vision deficiencies.
- Avoid telling users they definitely have color blindness based on a short online test.

My recommendation: If you're building this as a beginner web development project, start with just three main features: a color vision quiz, instant answer feedback, and a results page. You can add more advanced features later.

That would give you a useful, manageable first version of the website!

# Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

---

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

## 0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I need to use dataclasses, if statements, loops, maybe lists.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think defining the dataclasses and planning out what pages include what. I think creating the images for the tests will cause the most trouble because I either have to figure out how to import pictures or create the scatter plot images.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I think the pictures, creating the webpages/routes. I think I will implement the dataclasses first because my project will most likely rely heavily on them.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

I think other classes might interfere with how much time I have to work on the project, but I will plan when to get everything done to have time to work on the project. I will be using claude code to help me with complex coding I might not be able to complete by myself.

# 1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you

to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

# 2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

| **Requirement** | **Implementation** |
|---|---|
| **Purpose/Functionality**<br><br>A 1-2 sentence description of the purpose or functionality of your site | **I will be creating an online colorblind test to help users test if they are possibly colorblind and should get checked for colorblindness. This will test multiple types of colorblindness.** |
| **State Dataclass Attributes**<br><br>Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.<br><br>Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site. | user_name (str that stores the name of the user to personalize results, created when user takes test and resets when retaking the test), test_type (str that tracks what type of test the user wants to take, decides which plate will be used in the test), plates (list[Plate] which contains the plates for the test chosen, loops over the plates and only takes the plate that matches the test_type), current_index (int of which question the user is on, helping choose which plate to display and creates a progress bar, count is 0 at start test and retake, but goes up by 1 in each question), score (int of how many plates the user answered correctly, set to 0 when starting test and retaking it, goes up by 1 if the user's answer matches the actual answer), show_feedback (bool which allows user to choose if they want to see if they answered questions correctly, gets set in the start_test and used to decide if the user gets feedback or goes to the next question page) |
| **Second Dataclass**<br><br>Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified) | The dataclass is Plate<br>● image_file (str): this is the path to the dot image<br>● correct_answer (str): this will represent the number/symbol in the image<br>● test_type (str): this will be the type of colorblind test the user wants to take<br>● user_answer (str): starts as "" and is filled in |

| | |
|---|---|
| | with the user's answer, the results page will loop over this to show the user's answer vs the actual answer |
| **Routes** (minimum 7)<br><br>The names of your route functions | index, instructions, start_test, show_question, submit_answer, next_question, results, retake, learn_more |
| **Pages** (minimum 5)<br><br>A brief description of the pages in your site | Home: welcome page that lets user enter name, choose a test, and toggle on or off feedback<br>Instructions: describes what screen settings you should use to have accurate test results<br>Question: shows the test image with the number and box to type the symbol<br>Feedback: tells the user (if turned on) if they answered correctly<br>Results: shows the score and if they should see an eye doctor as they might be color blind, lists the user's answer next to the correct answer<br>Learn More: this will give users more information about colorblindness and can have links to eye care sites |
| **User Input** (minimum 3)<br><br>Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes<br><br>For more information on these input types, see Drafter Documentation. | TextBox: where the user enters their name for personalized results<br>TextBox: user types their answer here of what they see in the test<br>CheckBox: this allows the user to choose to see if they got the answer right<br>SelectBox: allows user to choose which test they want to take |
| **If Statements** (minimum 3)<br>Brief description of what will be evaluated in your if statements and how they will meaningfully | • If the answer box is empty the question page will show again instead of moving on with a pop up to enter an answer<br>• If the answer matches the correct answer the |

| | |
|---|---|
| change the logic of the site | score will go up<br>● If the feedback is on then the feedback pages will be shown<br>● If the last plate was shown then the user will go to the results page<br>● If the score is below a certain percent (80%) then the page will say to see a doctor |
| **Loop** (minimum 1)<br>This loop must iterate over the list/dict of dataclass instances in your State attribute.<br>Description of this loop and why it is meaningful to the site | My loop will loop over the plates: on the Results page it will loop through the plate in state.plate which will add a row showing the correct answer beside the user's answer so they can see what they missed, this will loop over all the plates |

# LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Can you help me figure out what attributes to use? State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.

Here are the State attributes I'd recommend. There are six, which gives you a cushion above the minimum of 4 in case one gets cut later.

| Attribute | Type | What it stores | How it's meaningfully modified |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `user_name` | `str` | The name the user enters on the Home page, used to personalize the Results page (“Elinor, you got 5/6 correct”). | Set in `start_test` from the name TextBox. Cleared in `retake`. |
| `test_type` | `str` | Which test the user picked: `"red-green"`, `"blue-yellow"`, or `"total"`. | Set in `start_test` from the SelectBox. It decides which plates get loaded. |
| `plates` | `list[Plate]` | The plates for the current test. This is your required list of instances of the second dataclass. | In `start_test`, it’s filled by looping over all plates and keeping the ones whose `test_type` matches. In `submit_answer`, the current plate’s `user_answer` is set to what the user typed. In `retake`, it’s emptied. |
| `current_index` | `int` | Which question the user is on. It’s used to show “Question 3 / 6” (your replacement for a progress bar) and to pick the plate to display. | Set to 0 in `start_test` and `retake`. Goes up by 1 in `next_question`. |

| | | | |
|---|---|---|---|
| `score` | `int` | How many plates the user answered correctly. | Set to 0 in `start_test` and `retake`. Goes up by 1 in `submit_answer` when the answer matches. The Results page uses it to decide whether to recommend an eye doctor. |
| `show_feedback` | `bool` | Whether the user wants to see “Correct!/Incorrect” after each question. | Set in `start_test` from the CheckBox. `submit_answer` uses it to decide between the Feedback page and the next question. |

### *Route overview*

| # | *Route function* | *Parameters (from inputs)* | *Page it shows* | *State it changes* |
|---|---|---|---|---|
| *1* | *`index`* | *none* | ***Home*** | *none* |
| *2* | *`instructions`* | *none* | ***Instructions*** | *none* |
| *3* | *`start_test`* | *`name`, `chosen_test`, `wants_feedback`* | *→ Question* | *`user_name`, `test_type`, `plates`, `current_index`, `score`, `show_feedback`* |
| *4* | *`show_question`* | *none* | ***Question*** | *none* |

| | | | | |
|---|---|---|---|---|
| 5 | `submit_answer` | `answer` | ***Feedback**, or → next question* | `plates` *(sets* `user_answer`*),* `score` |
| 6 | `next_question` | *none* | *→ Question or → Results* | `current_index` |
| 7 | `results` | *none* | ***Results*** | *none (it reads* `score` *and loops over* `plates`*)* |
| 8 | `retake` | *none* | *→ Home* | `user_name`*,* `plates`*,* `current_index`*,* `score` |
| 9 | `learn_more` | *none* | ***Learn More*** | *none* |

If Statements (minimum 3)

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site

can you describe these very briefly

- Correct answer (`submit_answer`): If the answer matches the plate's correct answer, the score goes up by 1.
- Feedback setting (`submit_answer`): If feedback is turned on, the Feedback page is shown. Otherwise the site skips to the next question.
- Last question (`next_question`): If that was the last plate, the user goes to the Results page. Otherwise they see the next question.
- Score threshold (`results`): If the score is below 80%, the site recommends seeing an eye doctor. Otherwise it says the results look normal.

That's 5, two more than the minimum of 3.

can you help me explain my iteration loop over the plates: Loop (minimum 1)

This loop must iterate over the list/dict of dataclass instances in your State attribute.

Description of this loop and why it is meaningful to the site

very briefly

Loop over `state.plates` (in `results`): The Results page loops through each plate in `state.plates` and adds a row showing the plate's `correct_answer` next to the user's `user_answer`. This is meaningful because it lets users see exactly which plates they missed, which can point to the type of color blindness they may have.

Use this loop for this section rather than the one in `start_test`. The `start_test` loop goes over `ALL_PLATES`, which is a global constant and not a State attribute, so it doesn't count for this requirement.

# LLM Conversation:

Can you help me code the website, leave first year coding material to me (and just direct me as if it was a homework problem with comments) and then can you do the hard stuff?

Ran 4 commands

I'll start with the hard part, the dot-image generator, then build `main.py` with the other parts left for you.

Ran 2 commands, created make_plates.py

Now let me check the images, including how they look through each color blindness simulation.

Ran a command, read check.png

Both colour-vision plates check out in simulation, though the blue-yellow ones keep a faint trace, which is fine for screening. I'm drafting a complete working version privately next, then I'll turn the other parts into TODOs for you.

Created main.py, ran 2 commands, 1 failed

Since Pyodide can't load here (its CDN is unreachable), I'll test the routes by calling them

directly from Python, as your unit tests will.

Ran the route functions directly to check the logic

The logic works, including a subtle bug I guarded against, explained in the file. Now the version for you:

Created main.py, ran a command

I've built the project in `/home/user/colorblind-test/`. The hardest parts are done, and the other parts are left as numbered TODOs (1–18) with homework-style hints.

Files

- `make_plates.py`: generates the 15 dotted test images, 5 for each test type. It's fully done and commented, since its techniques weren't covered in class. You don't need to run it again.
- `images/`: the 15 generated images.
- `main.py`: your website. I wrote the setup, all the plate data, the image display, the results table, the example tests and the server start. The TODOs are yours.

How the images work

- The images use published scientific simulations of each type of color blindness. The figure and background colors look different to normal vision but nearly identical to that type of color blindness.
- I checked this by running each image through those simulations. With normal vision every number shows. In simulation the test numbers disappear, except for faint traces on two blue-yellow plates.
- Plate 1 in each test is a "control" plate that everyone should read. It checks that the user's screen works and they understand the task.

Your TODOs

- Do TODOs 2 and 3 (the dataclasses) first. The file can't run without them.
- After that, they're mostly the if statements, the loops, setting state fields, building pages, and writing unit tests.

- Every route already has its signature, type hints and docstring, so you're filling in bodies.

What I tested

- A complete version of the routes (start → answer → feedback → next → results) works.
- Both files compile, and every line is under 80 characters.
- I couldn't load the site in a browser here: this sandbox blocks the server Drafter downloads its in-browser Python from. So the first real browser test will be yours.

Things to watch

- I used Drafter 2.2.1. If your class uses a different version and something like `Table(rows, header=[...])` errors, tell me and I'll adjust it.
- Use your own words for the page text, especially Learn More.
- Upload this whole conversation to your repo. The project rules require all AI conversations, and the comments marking which parts were "DONE FOR YOU" will help your TA see what you wrote.

If you get stuck on a TODO, paste what you tried and I'll give you a hint rather than the answer.

Created 2 files

make_plates.py
234 additions, 0 deletions
+234−0
main.py
304 additions, 0 deletions
+304−0
See all
538 additions, 0 deletions
+538−0

Can you help me with the page stuff, I am not great with it and would like to learn it better

A Drafter Page has two parts:

None

```
return Page(state, [ ...things to show on the screen... ])
```

1. `state`: always the first thing. It hands the current State to the next route.
2. A list `[ ]`: everything on the screen, top to bottom, separated by commas. It's a normal Python list, like `[1, 2, 3]`, except the items are things to display.

What can go in the list

| You write | What appears |
|---|---|
| `"Hello!"` | Plain text |
| `message` | Whatever text the variable holds |
| `Header("Title")` | Big bold heading text |
| `Button("Next", next_question)` | A button. Clicking it runs the route named `next_question` |
| `TextBox("answer")` | A box the user types into |
| `Image("images/a.png", 300, 300)` | A picture, 300 by 300 pixels |

A couple of rules:

- In a Button, the route name has no quotes and no parentheses: `next_question`, not `"next_question"` or `next_question()`. You're telling the button *which* route to run later, not running it now.
- Every item needs a comma after it. A trailing comma after the last item is fine.

Your feedback page, step by step

You need a heading, the message, and a Next button. Translate each into an item from the table:

- Heading → `Header("...")`, with whatever title you like
- The message → `message` (a variable, so no quotes)
- Next button → `Button("Next", next_question)`

how do I make a bulleted list?

You give `BulletedList` a normal Python list of strings, and Drafter puts a bullet point in front of each one.

Step 1: write a normal list of your tips.

```
None


["Turn your brightness up", "Do not zoom in"]
```

Step 2: put that list inside `BulletedList( )`.

```
None


BulletedList(["Turn your brightness up", "Do not zoom in"])
```

Step 3: put it in your Page list like any other item, with a comma after it.

```
None


return Page(state, [
   Header("Tips and Tricks"),
   BulletedList(["Turn your brightness up", "Do not zoom in"]),
   Button("Back", index),
])
```

On the page, that shows:

Tips and Tricks

- Turn your brightness up
- Do not zoom in

[Back]

With four tips the line gets long, so you can spread the list over several lines. It's the same code, just easier to read:

None

```
BulletedList([
   "Turn off night mode and/or blue light filters",
   "Turn your brightness up",
   "Sit about an arm's length from the screen",
   "Do not zoom in",
]),
```

So in your current code, take your four tip strings, put `BulletedList([` above them and `]),` below them, and you're done.

can you show me the state dataclass again so I can look at it while coding 8?

Here's the State dataclass you wrote:

None

```
@dataclass
class State:
   user_name: str
```

```
    test_type: str
    plates: list[Plate]
    current_index: int
    score: int
    show_feedback: bool
```

And here's what `start_test` receives from the Home page:

None

```
def start_test(state: State, name: str, chosen_test: str,
               wants_feedback: bool) -> Page:
```

For TODO 8, match them up:

| State field | Gets set to |
|---|---|
| `state.user_name` | `name` (from the TextBox) |
| `state.test_type` | `chosen_test` (from the SelectBox) |
| `state.show_feedback` | `wants_feedback` (from the CheckBox) |
| `state.score` | `0` |
| `state.current_index` | `0` |
| `state.plates` | `[]` (an empty list, filled by your loop in TODO 9) |

can you show me the plate dataclass for 15?

Here's your Plate dataclass:

None

```
@dataclass
class Plate:
   image_file: str      # path to the dotted image
   correct_answer: str  # the number hidden in the image
   test_type: str       # "red-green", "blue-yellow", or "total"
   user_answer: str     # what the user typed ("" until answered)
```

For TODO 15 you only need two of these: `plate.correct_answer` and `plate.user_answer`. By the time the Results page shows, `user_answer` holds what the user typed, or `""` if they left it blank.

can you also show me the state one?

Here's your State dataclass:

None

```
@dataclass
class State:
   user_name: str
   test_type: str
   plates: list[Plate]
   current_index: int
   score: int
   show_feedback: bool
```

For TODO 15 you loop over `state.plates`. It's a `list[Plate]`, so each item the loop gives you is one Plate with the fields above.

You'll also use `state.user_name`, `state.score` and `percent` for the "TODO" sentence on the Results page, the line that says something like "Elinor, you got 4 out of 5 correct."

```
    for plate in state.plates:
        answer_shown = plate.user_answer
        if answer_shown == "":
            answer_shown = "[blank]"
        rows.append([plate_number, plate.correct_answer, answer_shown])
        plate_number += 1
    name_shown = state.user_name
    if name_shown == "":
        name_shown = "You"
    score_message = (name_shown + ", you got " + str(state.score) + " out of "
                     + str(len(state.plates)) + " correct (" + str(percent) + "%).")
    return Page(state, [
        Header("Your Results"),
        score_message,
        recommendation,
        # DONE FOR YOU: turns your rows into a table with column titles.
        Table(rows, header=["Plate", "Correct answer", "Your answer"]),
        "This is a screening tool, not a medical diagnosis. Only an eye "
        "doctor can diagnose color blindness.",
        Button("Retake", retake),
        Button("Learn More", learn_more),
    ])
```

The `if` works, but with a blank name the page would say:

> You, you got 4 out of 5 correct (80.0%).

That "You, you" sounds odd. The easiest fix is to change only the replacement word, so nothing else needs to change:

if name_shown == "":

    name_shown = "Hey there"

Then a blank name gives "Hey there, you got 4 out of 5 correct (80.0%)." and a real name still gives "Elinor, you got 4 out of 5 correct (80.0%)."

Everything else is correct: the `if` is indented inside the function, it comes before `score_message`, and you used `name_shown` in the sentence.

This also gives you another meaningful `if` to point out in your video. You now have six: feedback correct/incorrect, score +1, show feedback or not, last question, eye doctor recommendation, and blank answer/name.

how do I make smaller headers

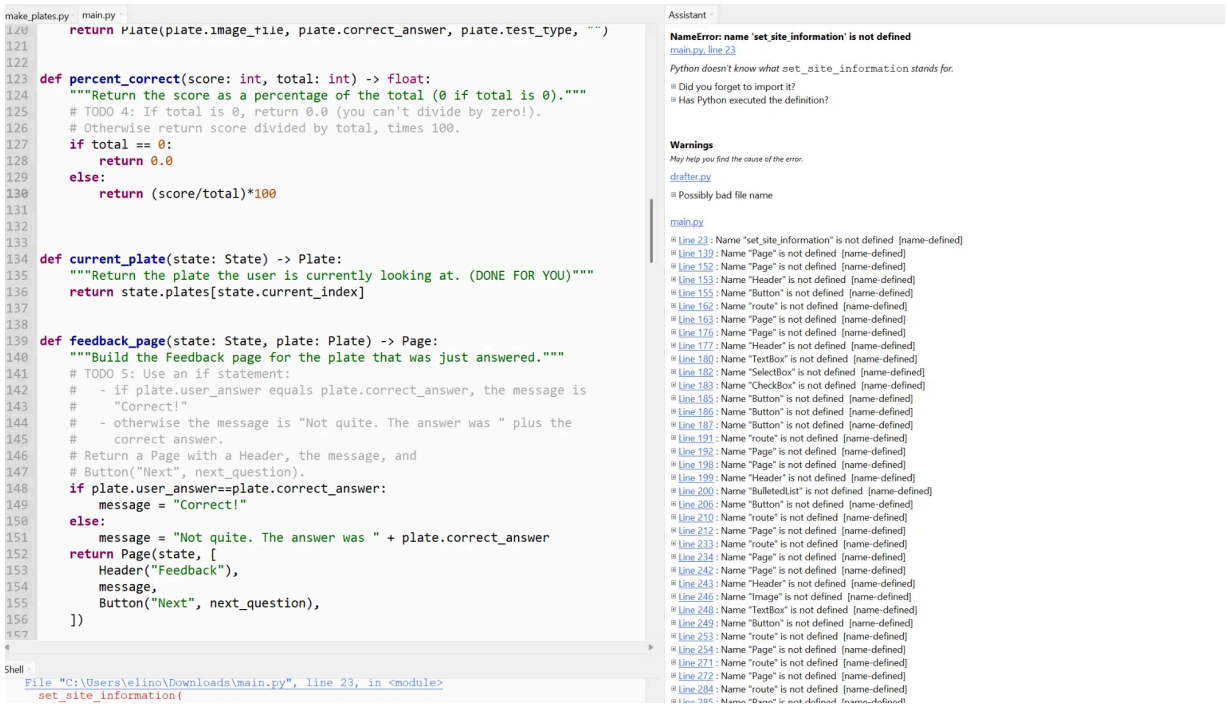
Add a number after the text in `Header`. The number sets the size: `1` is the biggest, and `6` is the smallest.

None

```
Header("Learn More About Color Blindness")      # level 1: biggest (default)
Header("Red-Green Color Blindness", 2)          # level 2: smaller
Header("Fun Fact", 3)                           # level 3: smaller still
```

If you leave the number out, it's level 1. A good habit is one level-1 Header per page for the page title, then level 2 for each section under it. That's what the Learn More skeleton uses, so you only need to type the `, 2` after the section titles.

can you help me with these errors?



make_plates.py × main.py ×

```
"""
Color Vision Check - a Drafter website that screens for red-green,
blue-yellow, and total color blindness using dotted test images.

HOW TO USE THIS FILE
- Parts marked "DONE FOR YOU" are the harder parts. Read the comments so
  you understand them (you'll explain your site in your video!).
- Parts marked "TODO" are yours, like a homework problem. Work top to
  bottom: the dataclasses MUST be done first or nothing else will run.
- The images were made by make_plates.py (already run; see that file).
"""
from drafter import *
from dataclasses import dataclass

# ---------------------------------------------------------------------------
# Global constants (allowed by the rules because they never change)
# ---------------------------------------------------------------------------
TEST_TYPES = ["red-green", "blue-yellow", "total"]
PASSING_PERCENT = 80   # below this, we recommend an eye doctor
```

Assistant ×

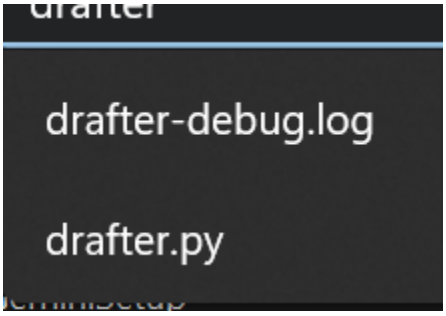
**NameError: name 'set_site_information' is not defined**
main.py, line 23

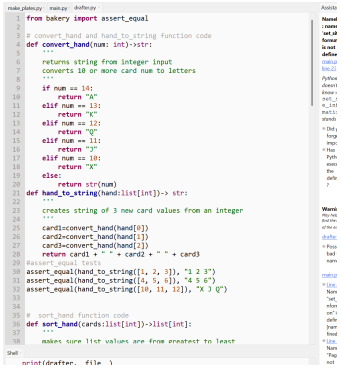
*Python doesn't know what* `set_site_information` *stands for.*

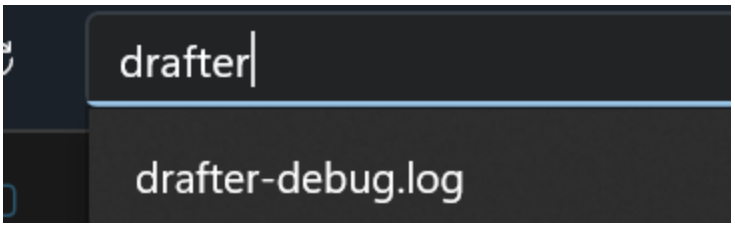
- Did you forget to import it?
- Has Python executed the definition ?

```
print(drafter.__file__)
```

```
C:\Users\elino\Downloads\drafter.py
```






(note from me - I did not realize the pics would not paste from claude and I am not exactly sure where they are in this specific mini convo so I just will put them all together here)

All these errors come from one problem. Every Drafter name (`Page`, `Header`, `Button`, `route`, `set_site_information`...) is "not defined," so Python isn't loading the real Drafter library.

The clue is the warning near the top of the panel:

drafter.py: Possibly bad file name

There's a file named `drafter.py` in the same folder as your `main.py` (your Downloads folder). When you write `from drafter import *`, Python checks your own folder first, finds that file, and imports it instead of the real library. That file doesn't contain `Page`, `Button` and the rest, hence all the errors.

How to fix it

1. Make a project folder. Create a new folder, for example `Documents\colorblind-test`, and move `main.py`, `make_plates.py` and the `images` folder into it. Downloads is risky because it collects random files.
2. Don't put any file named `drafter.py` in that folder. If the `drafter.py` in Downloads is something you need, rename it (for example `drafter_notes.py`). Otherwise delete it.
3. Open `main.py` from the new folder in Thonny and run it again.

If it still fails after that

- Check line 1 of `main.py`: it must say `from drafter import *`.
- Check that Drafter is installed in Thonny: Tools → Manage packages..., search for `drafter`, and click Install if it isn't already installed.

Also double-check

- The `images` folder must sit next to `main.py`, or the plate pictures won't show.
- You don't need to run `make_plates.py`. The images are already made, and it

needs extra libraries (Pillow and NumPy) that your site doesn't use.

Run it again after moving the files and send me the next error if there is one.

Line 12 has the correct import, which confirms the diagnosis. If Drafter weren't installed at all, you'd get a different error on line 12 ("No module named drafter"). Instead the import "works" but brings in nothing, so Python is still finding a wrong `drafter` somewhere first.

To see exactly which file it's loading, type these two lines into Thonny's Shell at the bottom, pressing Enter after each:

None

```
import drafter

print(drafter.__file__)
```

It prints the path of the file Python is actually using:

- If it shows a path in your project folder or Downloads, for example `C:\Users\elino\Downloads\drafter.py`, that's the impostor. Rename or delete that file. Also delete any `__pycache__` folder next to it, since that can hold a cached copy of the impostor.
- If it shows a folder named `drafter` (for example `...\Downloads\drafter\__init__.py`), a *folder* called `drafter` is causing the same problem. Rename that folder.
- If it shows a path with `site-packages` in it, the real library is being found. In that case, go to Tools → Manage packages, search `drafter`, and click Upgrade, because an old version may be missing `set_site_information`.

After fixing it, click Run → Stop/Restart backend (or the red Stop button) so Thonny forgets the old import, then run `main.py` again.

Tell me what the `print` line shows if you're not sure which case you're in.

That confirms it: Python is using `C:\Users\elino\Downloads\drafter.py`, a file in your Downloads folder, instead of the real Drafter library.

1. Open File Explorer and go to `C:\Users\elino\Downloads`.

2. Find `drafter.py`.
    - If you don't need it, delete it.
    - If you're not sure what it is, rename it to `drafter_old.py`.
3. If there's a `__pycache__` folder in Downloads, delete it too. It may hold a saved copy of the old file. It's safe to delete, because Python recreates it when needed.
4. Restart Thonny's backend: click the red Stop button, or Run → Stop/Restart backend.
5. Check again in the Shell:

None

6. 
7. `import drafter`
8. `print(drafter.__file__)`
9. 
    This time the path should contain `site-packages`. That's the real library.
10. Run `main.py`.

If step 5 instead gives "No module named drafter", the real library isn't installed. Go to Tools → Manage packages..., search `drafter`, and click Install.

To avoid this happening again, move your project into its own folder, like `Documents\colorblind-test` with `main.py`, `make_plates.py` and `images` inside, and never name your own files after libraries you import (`drafter.py`, `random.py`, `math.py`, and so on).

like the yellow folder app?

Yes, File Explorer is the yellow folder icon, usually on the taskbar at the bottom of your screen. You can also press Windows key + E to open it.

1. Open it, and click Downloads in the left sidebar.
2. Look for a file named `drafter`. Windows often hides the `.py` part, so it may just say `drafter`, with a Python icon or "Python File" in the Type column.
3. Right-click it, then:
    - Delete (the trash can icon), or
    - Rename, and change it to `drafter_old`.
4. If you see a folder named `__pycache__`, delete that too.

Downloads can get crowded. If it's hard to find, type `drafter` in the search box in the

top-right corner of the File Explorer window.

Then go back to Thonny, click the red Stop button, and run `main.py` again.

You found it. Those two items are search suggestions:

- `drafter.py`: the file causing the problem. Deal with this one.
- `drafter-debug.log`: a log file Drafter writes when it runs. It's harmless, so you can ignore it.

Before deleting, make sure this `drafter.py` is the one in Downloads:

1. Press Enter in the search box to see the full results. You should see `drafter.py` with its folder location listed.
2. Check that the location says `C:\Users\elino\Downloads`. If you're unsure, right-click it and choose Open file location.
3. Right-click `drafter.py` and choose Rename. Change it to `drafter_old.py`, or delete it if you know you don't need it.

Then go back to Thonny, click the red Stop button, and run your two Shell lines again:

None

```
import drafter

print(drafter.__file__)
```

You want the path to contain `site-packages` this time.

That file is one of your old homework assignments (the card hand functions), saved by accident with the name `drafter.py`. Don't delete it, just give it a new name.

1. Close the `drafter.py` tab in Thonny (click the small × on the tab). Windows may not let you rename a file that's open.
2. In File Explorer, right-click `drafter.py` in Downloads, choose Rename, and call it something like `card_hand.py`.
3. In Thonny, click the red Stop button to reset.
4. Run your check in the Shell again:

```
None
```

5.

```
6. import drafter
7. print(drafter.__file__)
```

8. You should now see a path with `site-packages` in it.
9. Run `main.py`.

This mix-up happened because both files live in Downloads. Once it works, move your final project into its own folder, such as `Documents\colorblind-test` with `main.py`, `make_plates.py` and the `images` folder, so it can't happen again.

The rename worked: `drafter.py` no longer shows up in the search, and the only match left is the harmless log file.

Now finish up in Thonny:

1. Click the red Stop button to reset Thonny.
2. Check in the Shell:

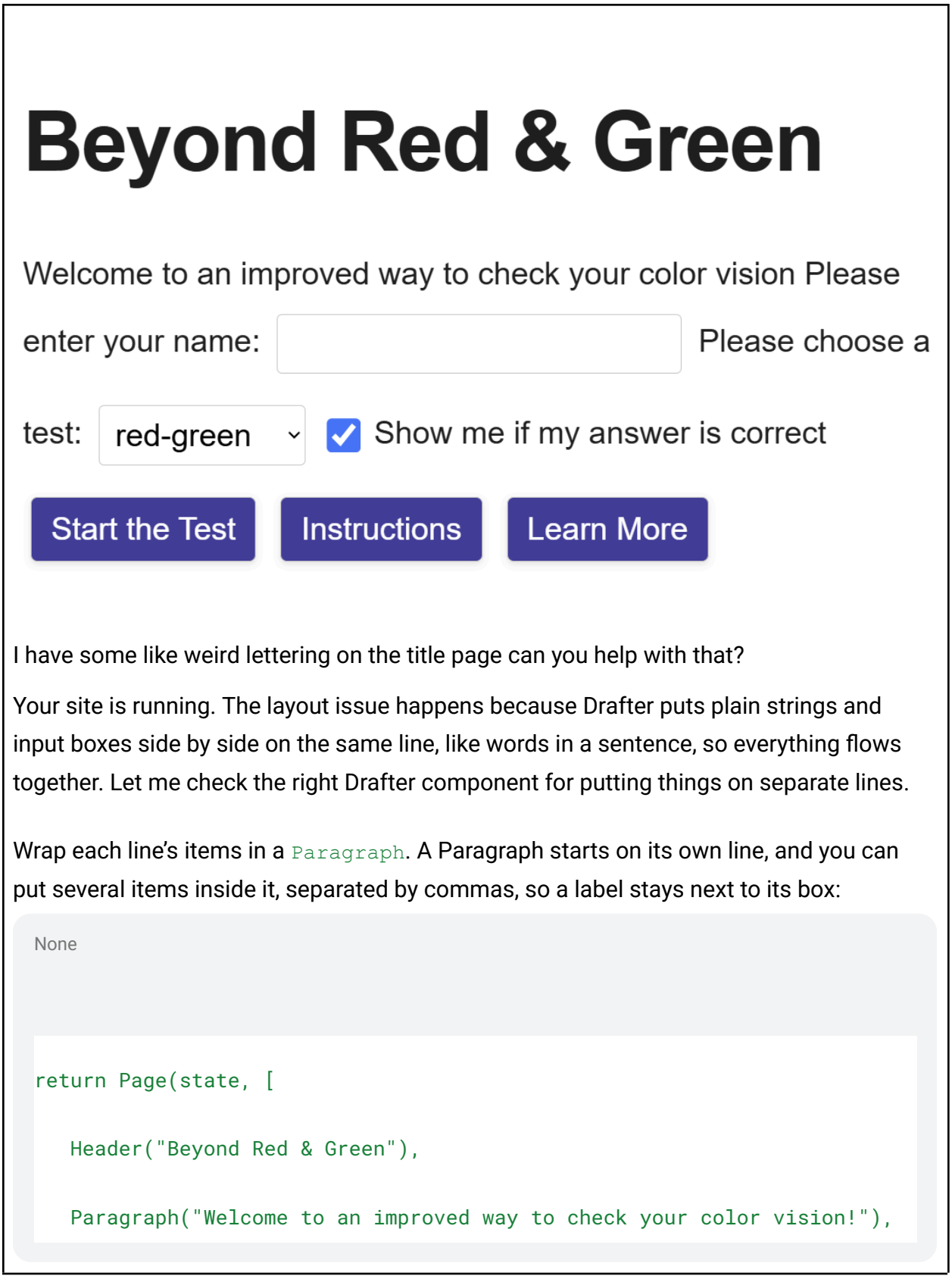
```
None
```

3.

```
4. import drafter
5. print(drafter.__file__)
```

- If the path contains `site-packages`, you're good. Run `main.py`.
- If it says "No module named drafter", the real library isn't installed yet. Go to Tools → Manage packages..., search `drafter`, click Install, then run `main.py`.

When you run `main.py`, your unit tests run first. Each one prints `SUCCESS` or shows what went wrong. Then the website should open in your browser. Send me any errors or failed tests.




I have some like weird lettering on the title page can you help with that?

Your site is running. The layout issue happens because Drafter puts plain strings and input boxes side by side on the same line, like words in a sentence, so everything flows together. Let me check the right Drafter component for putting things on separate lines.

Wrap each line's items in a `Paragraph`. A Paragraph starts on its own line, and you can put several items inside it, separated by commas, so a label stays next to its box:

None

```
return Page(state, [
  Header("Beyond Red & Green"),
  Paragraph("Welcome to an improved way to check your color vision!"),
```

```
   Paragraph("Please enter your name:", TextBox("name",
state.user_name)),
   Paragraph("Please choose a test:", SelectBox("chosen_test",
TEST_TYPES)),
   Paragraph(CheckBox("wants_feedback", True),
             "Show me if my answer is correct"),
   Button("Start the Test", start_test),
   Button("Instructions", instructions),
   Button("Learn More", learn_more),
])
```

That shows:

Beyond Red & Green

Welcome to an improved way to check your color vision!

Please enter your name: [_____]

Please choose a test: [red-green ▾]

☑ Show me if my answer is correct

[Start the Test] [Instructions] [Learn More]

What changed:

- `Paragraph(...)` uses round parentheses, not square brackets, with the items inside separated by commas.
- Each Paragraph is still one item in the Page list, so it needs a comma after it.
- I also added a `!` after “color vision” so it doesn’t run into the next sentence.

You can use the same fix on other pages that look squished. For example, on the Question page, wrap `question_count` and the “Type the number...” line in `Paragraph(...)`. On the Results page, wrap `score_message`, `recommendation` and the disclaimer.

One thing to watch: if you wrap `question_count` in a Paragraph, your unit test `assert_equal(test_page.content[1], "Question 1 / 5")` will fail, because item `[1]` is now a Paragraph instead of a plain string. Update that test to:

None

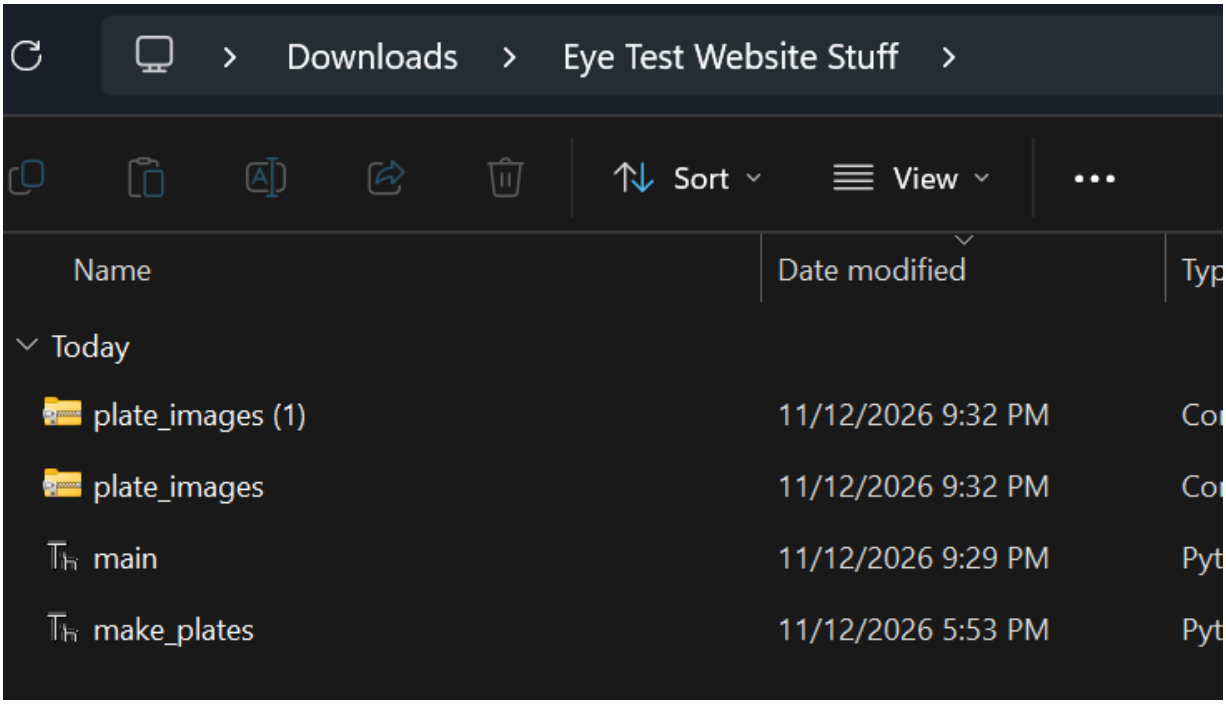
```
assert_equal(test_page.content[1], Paragraph("Question 1 / 5"))
```

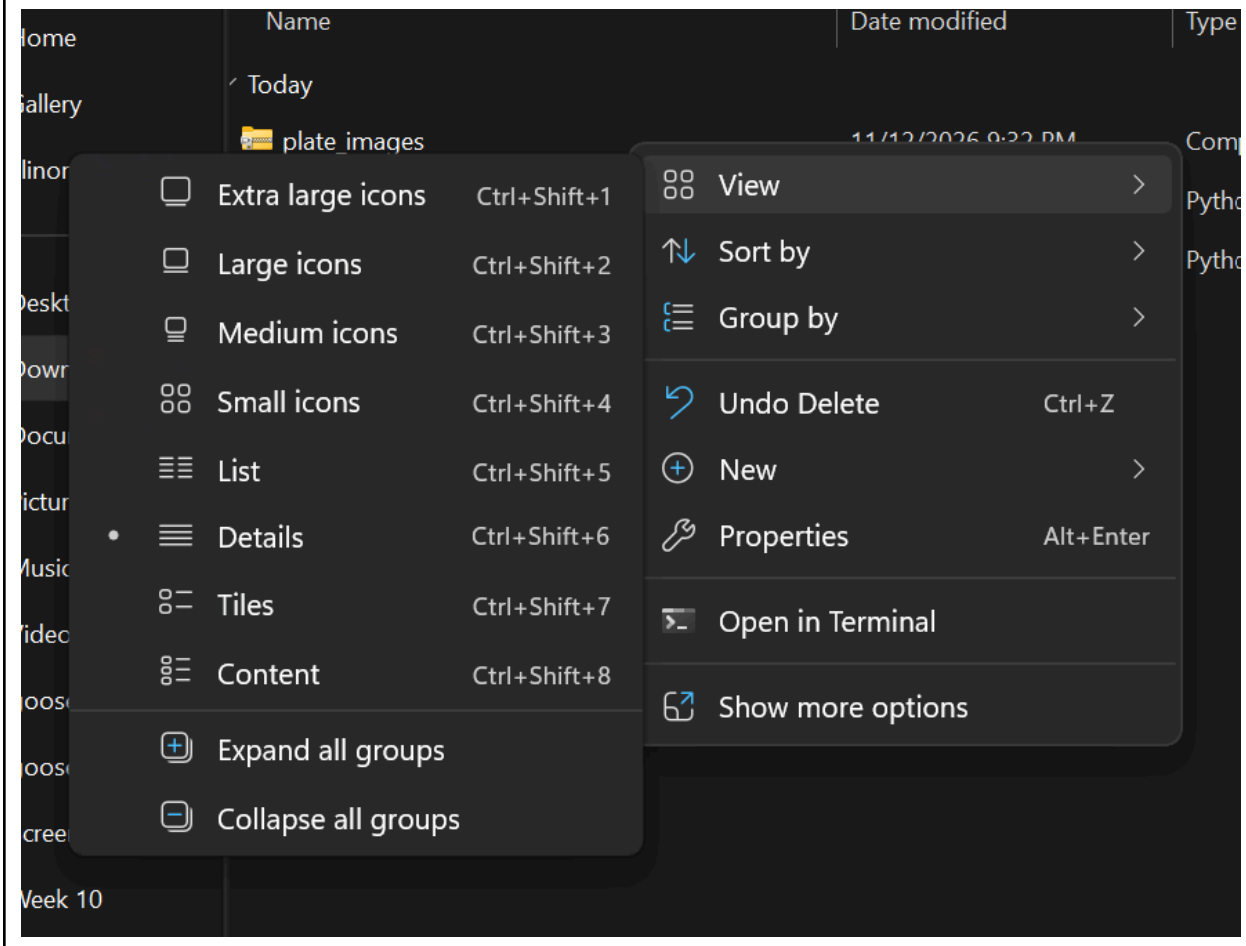
The same applies to your `score_message` test on the Results page.

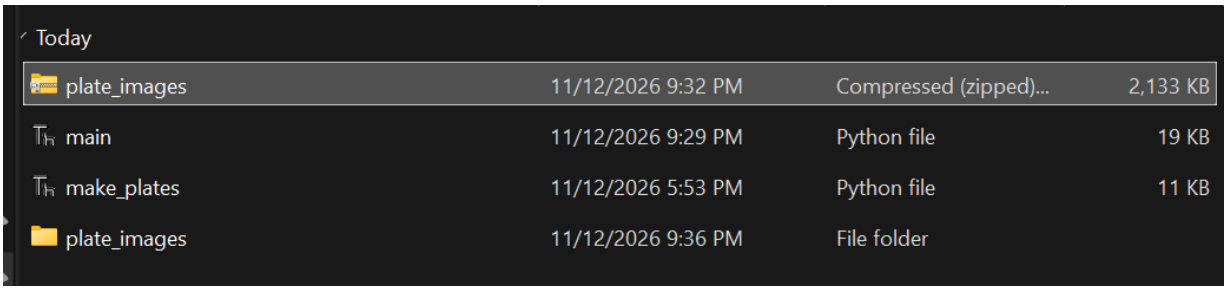
Drafter Application Navigate View Edit Save/Load

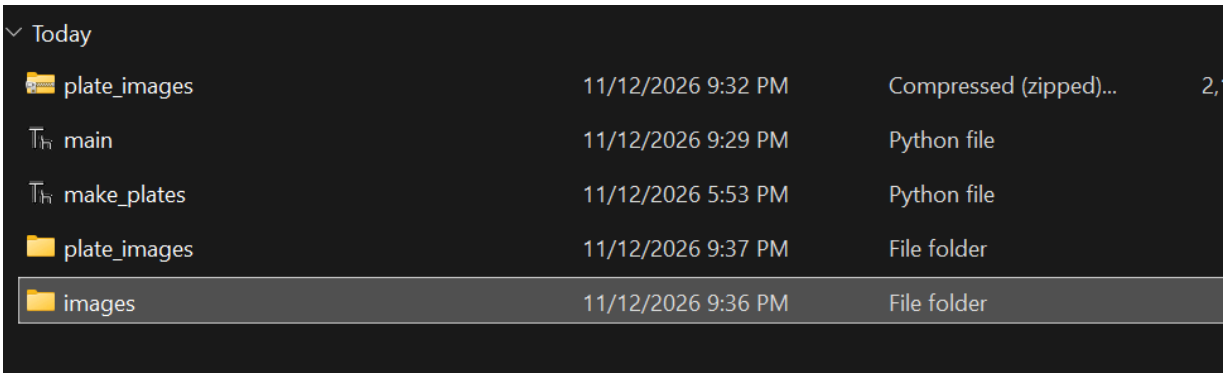
# What number do you see?

Question 1 / 5  Type the number (leave it blank if you can't see one): Submit




Name
Date modified
Type
Today
plate_images
Extra large icons Ctrl+Shift+1
Large icons Ctrl+Shift+2
Medium icons Ctrl+Shift+3
Small icons Ctrl+Shift+4
List Ctrl+Shift+5
Details Ctrl+Shift+6
Tiles Ctrl+Shift+7
Content Ctrl+Shift+8
Expand all groups
Collapse all groups
View
Sort by
Group by
Undo Delete Ctrl+Z
New
Properties Alt+Enter
Open in Terminal
Show more options


Today
plate_images 11/12/2026 9:32 PM Compressed (zipped)... 2,133 KB
main 11/12/2026 9:29 PM Python file 19 KB
make_plates 11/12/2026 5:53 PM Python file 11 KB
plate_images 11/12/2026 9:36 PM File folder




for some reason the image did not pop up

A broken image box means the browser can't find the picture file. The code is fine; the file just isn't where `main.py` expects it. `Image(plate.image_file, ...)` looks for images/red_green_1.png, which means a folder named `images` sitting right next to `main.py`.

Your `main.py` is in Downloads, so check in File Explorer that you have:

None

```
Downloads\
   main.py
   images\
       red_green_1.png
       red_green_2.png
       ... (15 images total)
```

You might not have the images at all yet if you only copied the code over, so I'll package them into one zip file for you:

All 15 test plate images, already inside an "images" folder.

zip

plate_images.zip

plate_images.zip

2.2MB

The image boxes are empty because the browser can't find the picture files. Your code is fine; the files just aren't where `main.py` expects them. It looks for a folder named `images` right next to `main.py`.

I've sent you `plate_images.zip`, which has all 15 test images already inside an `images` folder. To put them in place:

1. Download `plate_images.zip`. It will probably land in Downloads.
2. Right-click it, choose Extract All..., and set the destination to the folder that holds `main.py`.
    - Windows may suggest a new folder called `plate_images`. Delete that part of the path so the files go straight into your project folder.
3. Check in File Explorer that the layout looks like this, with `images` directly beside `main.py`:

None

4. 
5. `main.py`
6. `images\`
7. `    red_green_1.png`
8. `    red_green_2.png`
9. `    ... (15 images)`
10. If you instead see `images\images\...` or `plate_images\images\...`, move the inner `images` folder up so it sits next to `main.py`.
11. In Thonny, press Stop, then run `main.py` again.

While you're on the Question page: the question count, the image and the answer box are all squished onto one line, like the Home page was. Wrap them in `Paragraph(...)` the same way. If you wrap `question_count`, update its unit test to `Paragraph("Question 1 / 5")`, as I mentioned before.

this is what I did

You're close. You've downloaded the zip into the right folder (twice), but it hasn't been unzipped yet. The zipper icon on those folders and "Compressed" in the Type column mean they're still zip files. The website can't read pictures inside a zip.

1. Delete `plate_images (1)`. It's just a duplicate download.
2. Right-click `plate_images` and choose Extract All...
3. In the box that pops up, the path will end with something like `...\Eye Test Website Stuff\plate_images`. Delete the `\plate_images` part at the end, so it reads `...\Eye Test Website Stuff`.
4. Click Extract.

You should then see a regular yellow folder named `images` (no zipper) next to `main` and `make_plates`:

None

```
Eye Test Website Stuff\
  images          ← new, regular folder with 15 pictures
  main
  make_plates
  plate_images    ← the zip (you can delete it after extracting)
```

5. In Thonny, make sure you have `main.py` open from this folder (`Eye Test Website Stuff`), not an older copy in Downloads. Press Stop, then run it.

Moving the project into its own folder was a good idea too. It keeps it safe from the `drafter.py` mix-up.

That's the menu for the empty space in the folder, so it doesn't have the Extract option. You need to right-click directly on the file.

1. Press Esc to close this menu.
2. Right-click directly on the words `plate_images` (the one with the zipper icon).
3. Choose Extract All... If you don't see it, click Show more options at the bottom of that menu, and it'll be in the longer list.

If that's still tricky, another way:

1. Double-click `plate_images` to open it. Inside you'll see a folder named `images`.
2. Drag that `images` folder up into the Eye Test Website Stuff path at the top of the window (or press the Back arrow and paste it there).

Either way, you should end up with a regular yellow `images` folder, without a zipper, sitting next to `main` and `make_plates`.

when I right click it won't come up

Try one of these instead.

Option 1: the toolbar button

1. Click once on `plate_images` (the zip) to select it, so it's highlighted.
2. Look at the toolbar at the top of the window. An Extract all button should appear there. If you don't see it, click the `...` (three dots) on the toolbar.
3. Click Extract all, delete the `\plate_images` part at the end of the path, and click Extract.

Option 2: copy and paste

1. Double-click `plate_images` to open it like a folder. You'll see a folder named `images` inside.
2. Click once on `images`, then press Ctrl + C to copy it.
3. Click the Back arrow (top-left) to return to Eye Test Website Stuff.

4. Click an empty spot and press Ctrl + V to paste.

Either way, you should end up with a yellow `images` folder, without a zipper, next to `main` and `make_plates`. Double-click it to check that the 15 pictures are inside, then run `main.py` again in Thonny.

The extraction worked, but it put everything in a new folder called `plate_images`. The `images` folder is now *inside* that, one level too deep:

None

```
Eye Test Website Stuff\
   plate_images\       ← regular folder (the new one)
       images\         ← needs to move up one level
   main
```

To move it up:

1. Double-click the regular `plate_images` folder, the bottom one with the plain yellow icon and no zipper.
2. Click once on the `images` folder inside, then press Ctrl + X to cut it.
3. Click the Back arrow to return to Eye Test Website Stuff.
4. Click an empty spot and press Ctrl + V to paste.

You should then see:

None

```
images          ← folder, next to main ✓
main
make_plates
plate_images    (zip, can delete)
```

```
plate_images    (folder, now empty, can delete)
```

Then you can delete both `plate_images` items to tidy up, and run `main.py` in Thonny again.

The `images` folder is now right next to `main`, exactly where it needs to be.

1. Optional tidy-up: delete both `plate_images` items (the zip and the now-empty folder). You don't need them anymore.
2. In Thonny, make sure the `main.py` you have open is the one in Eye Test Website Stuff. You can check by hovering over the tab or looking at the path in the Shell when it runs.
3. Press Stop, then Run `main.py`.

The colored dot images should show up now. If you still see an empty box, send me a screenshot of the Shell output and I'll help track it down.